



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки Кафедра
інформаційних систем та технологій

Лабораторна робота №6
із дисципліни «**Технології розроблення програмного забезпечення**»
Патерни проектування.

Виконав:

студент групи ІА–34

Оборський Дмитро Вікторович

Київ 2025

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

14. Архіватор (strategy, adapter, factory method, facade, visitor, p2p)

Архіватор повинен являти собою візуальний додаток з можливістю створення і редагування архівів різного типу (.tar.gz, .zip, .rar, .ace) – додавання/ видалення файлів / папок, редагування метаданих (по можливості), перевірка checksum архівів, тестування архівів на наявність пошкоджень, розбиття архівів на частини.

Теоретичні відомості

1.1.1. Шаблон «Abstract Factory»

Призначення патерну: Шаблон «Абстрактна фабрика» використовується для створення сімейств об'єктів без вказівки їх конкретних класів [6]. Для цього виноситься загальний інтерфейс фабрики (AbstractFactory) і створюються його реалізації для різних сімейств продуктів. Хорошим прикладом використання абстрактної фабрики є ADO.NET: існує загальний клас DbProviderFactory, здатний створювати об'єкти типів DbConnection, DbDataReader, DbAdapter та ін.; існують реалізації цих фабрик і об'єктів – SqlProviderFactory, SqlConnection, SqlDataReader, SqlAdapter і так далі. Відповідно, якщо додатку необхідно працювати з різними базами даних (чи потрібна така можливість), то досить

використати базові реалізації (Db.) і підставити відповідну фабрику у момент ініціалізації фабрики (Factory = new SqlProviderFactory()).

Цей шаблон передусім структурує знання про схожі об'єкти (що називаються сімействами, як класи для доступу до БД) і створює можливість взаємозаміни різних сімейств (робота з Oracle ведеться також, як і робота з SQL Server). Проте, при використанні такої схеми у край незручно розширювати фабрику – для додавання нового методу у фабрику необхідно додати його в усіх фабриках і створити відповідні класи, що створюються цим методом.

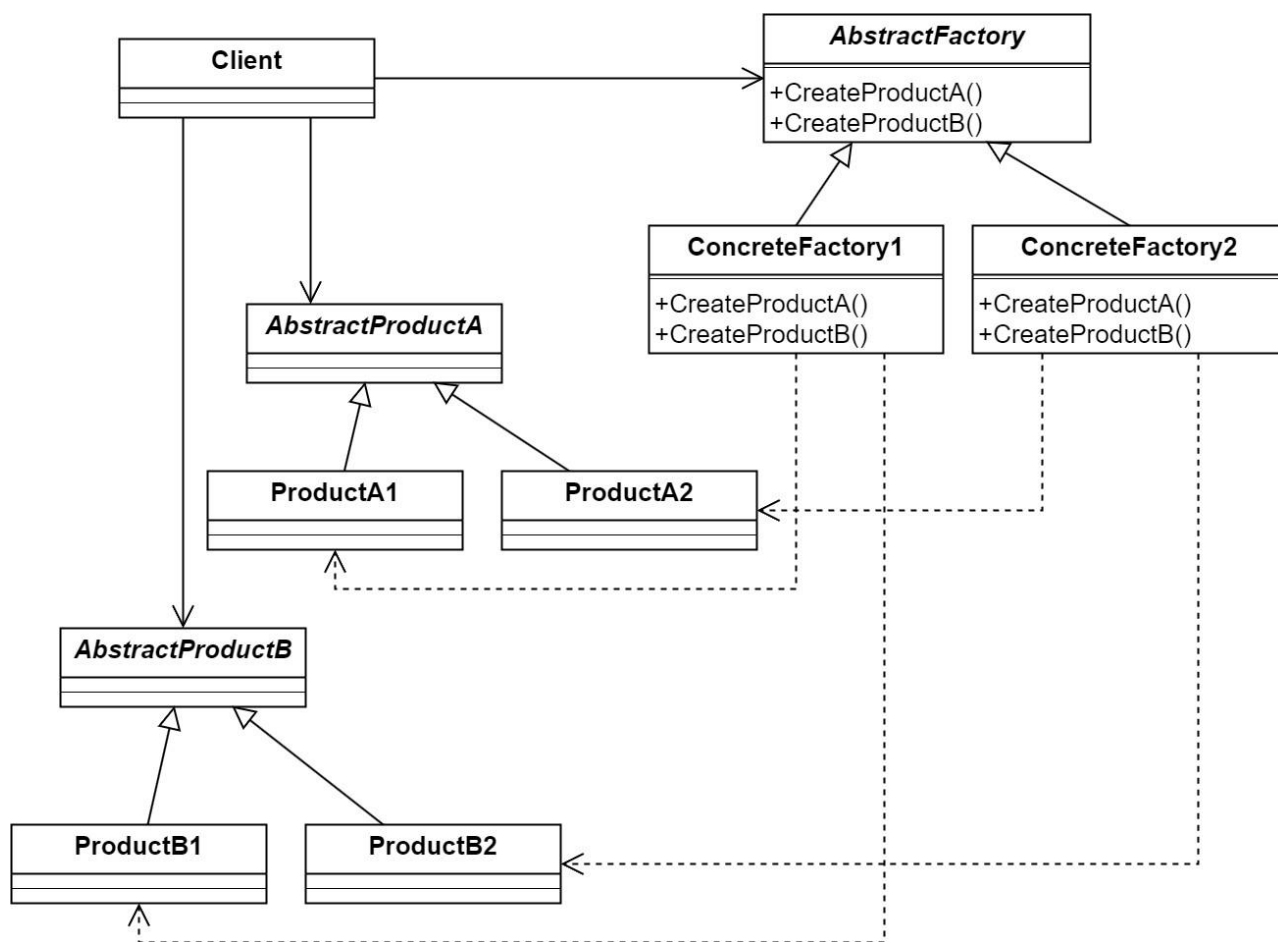


Рисунок 6.1. Структура патерну «Абстрактна фабрика»

Проблема: Ви розробляєте для Roguelike гри модуль автоматичної генерації кімнат. Модуль генерації кімнат, в процесі генерації конкретної кімнати, створює стіни, двері, вікна, підлогу та меблі в кімнаті. Модуль повинен підтримувати генерацію кімнат у різних стилях, таких як стиль хайтек, модерн,

класичний офіс. Також критично є щоб всі елементи в одній згенерованій кімнаті відносилися до одного стилю.

Рішення: Для вирішення поставленої задачі дуже добре підходить використання патерну «Абстрактна фабрика».

Для кожного типу продукту створюємо свій інтерфейс продукту який об'являє функції доступні для використання з цим продуктом. Наприклад, можна виділити інтерфейси для стін, вікон, дверей, підлоги, а також окремі інтерфейси для меблів, такі як стіл, шафа, крісло та інші. Від кожного інтерфейсу наслідуюмо і реалізуємо продукти різних типів, наприклад, для стола: інтерфейс `ITable` та реалізації продуктів `HighTechTable`, `ModernTable` та інші.

Далі визначаємо інтерфейс для абстрактної фабрики, який буде містити методи для створення кожного типу продукту. Створюємо класи конкретних фабрик під кожен стиль: `HighTechFabric`, `ModernFabric` та інші. Кожна конкретна фабрика буде створювати всі типи продуктів, але всі вони будуть відноситися до одного стилю.

Далі в алгоритм генерації кімнати будемо передавати конкретну фабрику і алгоритм при створенні продуктів тільки через фабрику завжди буде отримувати продукти одного стилю і працювати з продуктами тільки через інтерфейс продукта.

Таким чином ми вирішуємо проблему узгодженості стилів всіх елементів в кімнаті, а за рахунок використання різних конкретних фабрик ми зможемо генерувати кімнати різних стилів. Якщо нам потрібно буде додати ще один стиль, то достатньо буде реалізувати нові дочірні класи для кожного елемента кімнати, а також нову конкретну фабрику під цей стиль.

Переваги та недоліки:

- + Спрощує створення об'єктів і код стає легшим для розуміння.
- + Об'єкти створені однією фабрикою добре узгоджуються один з одним і зменшується кількість помилок взаємодії між ними.
- + Відокремлення створення об'єктів від їх використання, за рахунок чого, код стає більш структурованим.

- + Додавання нових сімейств продуктів виконується без зміни існуючого коду.
- Збільшується складність коду, особливо для простих проєктів.
- Додавання нового типу продукту є складним і вимагає змін коду в багатьох місцях.

1.1.2. Шаблон «Factory Method»

Призначення: Шаблон «Фабричний метод» визначає інтерфейс для створення об'єктів певного базового типу [6]. Це зручно, коли хочеться додати можливість створення об'єктів не базового типу, а деякого дочірнього. Фабричний метод у такому разі є зачіпкою для впровадження власного конструктора об'єктів. Основна ідея полягає саме в заміні об'єктів їх підтипами, що при цьому зберігає ту ж функціональність; інша частина поведінки об'єктів не є інтерфейсною (AnOperation) і дозволяє взаємодіяти із створеними об'єктами як з об'єктами базового типу. Тому шаблон «Фабричний метод» носить ще назву «Віртуальний конструктор».

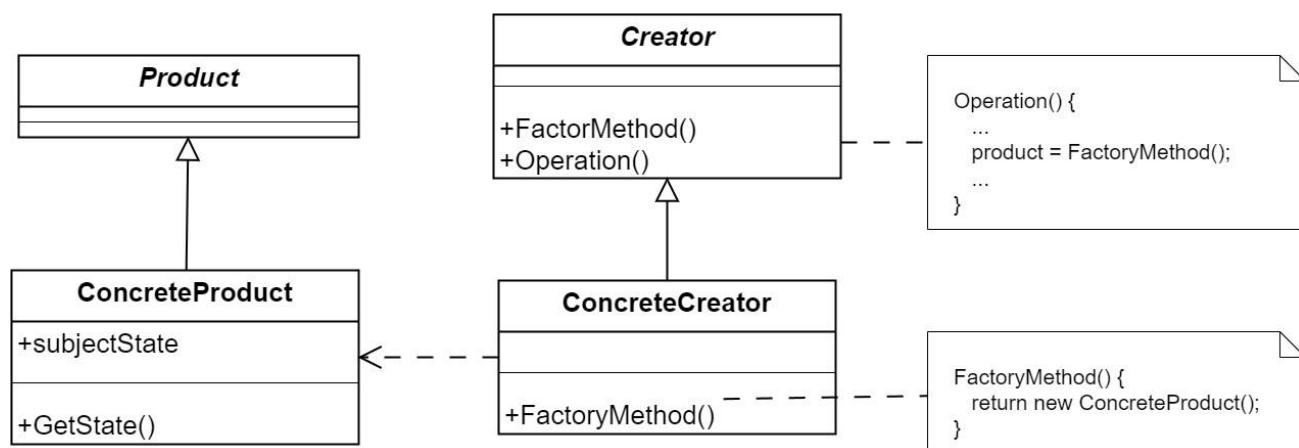


Рисунок 6.2. Структура патерну «Фабричний Метод»

Розглянемо простий приклад. Нехай наш застосунок працює з мережевими драйвер-мі і використовує клас `Packet` для зберігання даних, що передаються в мережу. Залежно від використовуваного протоколу, існує два перевантаження —

TcpPacket, UdpPacket. І відповідно два створюючі об'єкти (TcpCreator, UdpCreator) з фабричним методом (який створює відповідні реалізації).

Проте базова функціональність (передача пакету, прийом пакету, заповнення пакету даними) нічим не відрізняється один від одного, відповідно поміщається у базовий клас PacketCreator. Таким чином поведінка системи залишається тим же, проте з'являється можливість підстановки власних об'єктів в процес створення і роботи з пакетами.

Переваги та недоліки:

- + Позбавляє клас від прив'язки до конкретних класів продуктів.
- + Виділяє код виробництва продуктів в одне місце, спрощуючи підтримку коду.
- + Спрощує додавання нових продуктів до програми.
- Може призвести до створення великих паралельних ієрархій класів.

1.1.3. Шаблон «Memento»

Призначення: Шаблон використовується для збереження і відновлення стану об'єктів без порушення інкапсуляції [6]. Об'єкт «Memento» служить виключно для збереження змін над початковим об'єктом (Originator). Лише початковий об'єкт має можливість зберігати і отримувати стан об'єкту «Memento» для власних цілей, цей об'єкт є «порожнім» для кого-небудь ще. Об'єкт «Caretaker» використовується для передачі і зберігання мemento об'єктів в системі.

Таким чином вдається досягти наступних цілей:

- зберігання стану повністю відділяється від початкових об'єктів, що полегшує їх реалізацію;
- передача об'єктів «Memento» лягає на плечі Caretaker об'єктів, що дозволяє гнучкіше управляти станами об'єктів і спростити дизайн класів початкових об'єктів;

- збереження і відновлення стану реалізовані у вигляді двох простих методів і є закритими для кого-небудь ще окрім початкових об'єктів, таким чином не порушуючи інкапсуляцію.

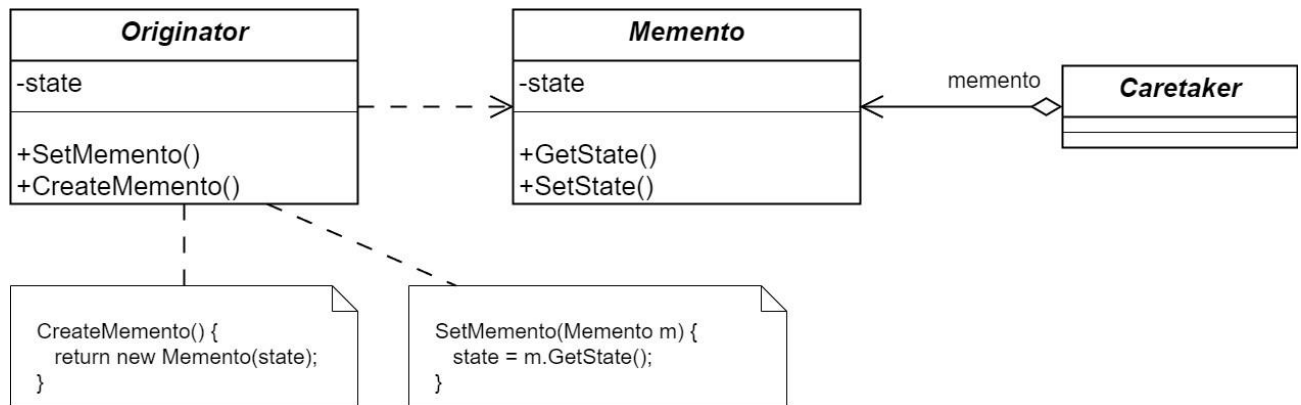


Рисунок 6.3. Структура шаблону «Знімок»

Шаблон «Мементо» дуже зручно використати разом з шаблоном «Команда» для реалізації «скасовних» дій – дані про дію зберігаються в мементо, а команда має можливість вважати і відновити початкове положення відповідних об'єктів.

Переваги та недоліки:

- + Не порушує інкапсуляцію вихідного об'єкта.
- + Спрощує структуру вихідного об'єкта. Не потрібно зберігати історію версій свого стану.
- Вимагає багато пам'яті, якщо клієнти дуже часто створюють знімки.
- Може спричинити додаткові витрати пам'яті, якщо об'єкти, що зберігають історію, не звільняють ресурси, зайняті застарілими знімками.

1.1.4. Шаблон «Observer»

Призначення: Шаблон визначає залежність «один-до-багатьох» таким чином, що коли один об'єкт змінює власний стан, усі інші об'єкти отримують про це сповіщення і мають можливість змінити власний стан також [6].

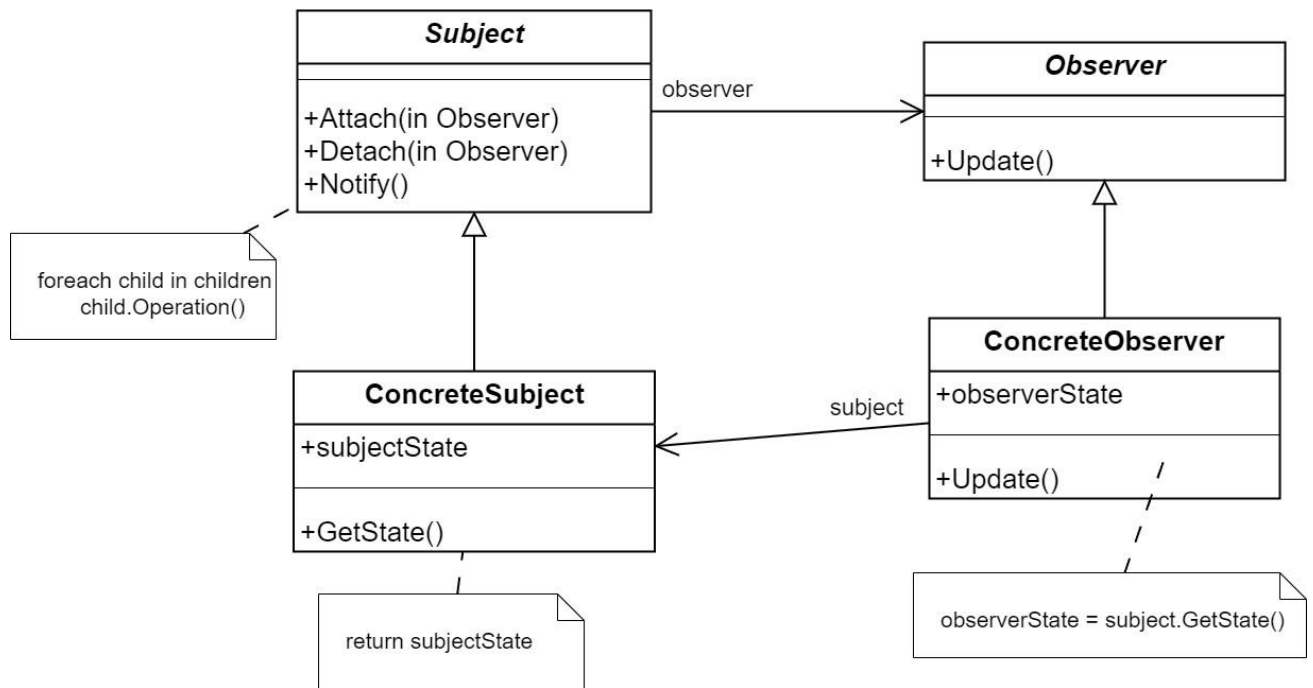


Рисунок 6.4. Структура патерна «Спостерігач»

Розглянемо цей шаблон на прикладі. Припустимо, є деяка банківська система і декілька користувачів переглядають баланс на рахунку пана І. У цей момент пан І. кладе на свій рахунок деяку суму, яка міняє загальний баланс. Кожен з користувачів, що переглядали баланс, отримує про це звістку (для користувачів ця звістка може бути прозорою – просто зміна цифр, або попередження про те, що баланс змінився). Раніше неможливі дії для користувачів (переведення до іншої категорії клієнтів і тому подібне) стають доступними.

Цей шаблон дуже широко поширений в шаблоні MVVM і механізмі «прив'язок» (bindings) в WPF і частково в WinForms. Інша назва шаблону – підписка/розсилка. Кожен з оглядачів власноручно підписується на зміни конкретного об'єкту, а об'єкти зобов'язані сповіщати своїх передплатників про усі свої зміни (на даний момент конкретних механізмів автоматичного сповіщення про зміну стану в .NET мовах не існує).

Приклад з життя: Коли ви підписуєтеся на канал на YouTube і натискаєте на «дзвіночок» ви фактично оформлюєте підписку на отримання повідомлень про вихід нових відео. Вам не потрібно заходити на канал і перевіряти чи не

вийшло нове відео. Ви будете отримувати повідомлення про вихід нових відео на каналах

Фактично на сервісі YouTube є список підписників на канал, хто має отримувати повідомлення про вихід нового відео.

В якості прикладу також можна згадати підписку «Повідомити про появу товару» на сторінці товару в магазині «Розетка».

Переваги та недоліки:

- + Можливість паралельної та асинхронної обробки повідомлень про оновлення.
- + Спостерігачів можна добавляти та видаляти в будь-який момент часу.
- + Спостерігач і суб'єкт можуть працювати в різних потоках.
- + Реалізує принцип слабого зв'язку між об'єктами.
- Послідовність розсилки повідомлень підписникам не підтримується.

1.1.5. Шаблон «Decorator»

Призначення: Шаблон призначений для динамічного додавання функціональних можливостей об'єкту під час роботи програми [6]. Декоратор деяким чином «обертає» (за рахунок агрегації) початковий об'єкт зі збереженням його функцій, проте дозволяє додати додаткові дії. Такий шаблон надає гнучкіший спосіб зміни поведінки об'єкту чим просте спадкоємство, оскільки початкова функціональність зберігається в повному об'ємі. Більше того, таку поведінку можна застосовувати до окремих об'єктів, а не до усієї системи в цілому.

Простим прикладом є накладення смуги прокрутки до усіх візуальних елементів. Кожен об'єкт, який може прокручуватися, обертається в «прокручуваному» елементі, і при необхідності з'являється полоса прокрутки. Початкові функції елементу (наприклад, рядки статусу) залишаються незмінними.

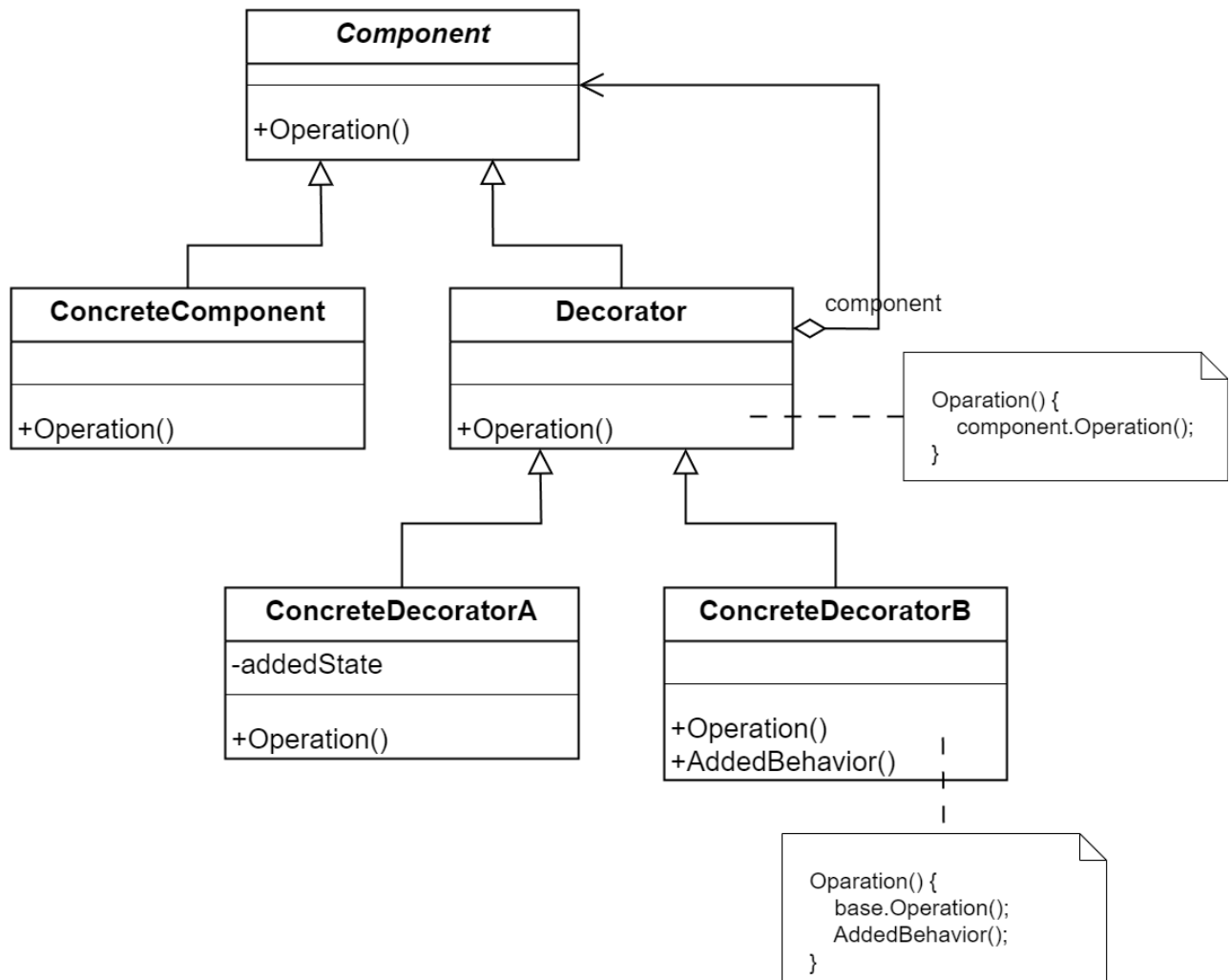


Рисунок 6.5. Структура патерну «Декоратор»

Переваги та недоліки:

- + Дозволяє мати кілька дрібних об'єктів, замість одного об'єкта «на всі випадки життя».
- + Дозволяє додавати обов'язки «на льоту».
- + Більша гнучкість, ніж у спадкування.
- Велика кількість крихітних класів.
- Важко конфігурувати об'єкти, які загорнуто в декілька обгортки одночасно.

Хід роботи

1. Створюю ArchiverFactory з відповідними стратегіями

```
1 package com.beowulf.core.factory;
2
3 import com.beowulf.core.interfaces.Archiver;
4 import com.beowulf.core.strategy.*;
5
6 import java.nio.file.Path;
7 import java.util.Locale;
8
9 You, 3 days ago | 1 author (You)
10 public class ArchiverFactory {
11     /**
12      * Returns an Archiver strategy based on the file extension.
13      */
14     public Archiver getArchiver(Path archive) {
15         String name = archive.getFileName().toString().toLowerCase(Locale.ROOT);
16
17         if (name.endsWith(".tar.gz") || name.endsWith(".tgz")) {
18             return new TarGzArchiver();
19         }
20
21         if (name.endsWith(".zip")) {
22             return new ZipArchiver();
23         }
24
25         if (name.endsWith(".rar")) {
26             return new RarArchiver();
27         }
28
29         if (name.endsWith(".ace")) {
30             return new AceArchiver();
31         }
32
33         throw new IllegalArgumentException("Unsupported archive format");
34     }
35 }
36 }
```

Рис. 1 ArchiverFactory

2. Створюю декоратор ArchiverLogger для логування

```
1 package com.beowulf.core.decorator;
2
3 import com.beowulf.core.interfaces.Archiver;
4 import com.beowulf.core.service.ArchivePersistenceService;
5 import com.beowulf.core.user.AppUser;
6 import com.beowulf.core.user.AppUserService;
7 import com.beowulf.core.utils.ArchiveMetadataUtil;
8 import com.beowulf.core.visitor.ArchiveOperation;
9 import com.beowulf.core.visitor.ArchiveVisitor;
10
11 import java.io.IOException;
12 import java.nio.file.Files;
13 import java.nio.file.Path;
14 import java.util.concurrent.TimeUnit;
15
16 public class ArchiverLogger implements Archiver {
17     private final Archiver delegate;
18     private final AppUserService identityProvider;
19     private final ArchiveVisitor visitor;
20     private final String operationOverride;
21
22     public ArchiverLogger(Archiver delegate,
23                          AppUserService identityProvider,
24                          ArchiveVisitor visitor) {
25         this(delegate, identityProvider, visitor, operationOverride: null);
26     }
27
28     public ArchiverLogger(Archiver delegate,
29                          AppUserService identityProvider,
30                          ArchivePersistenceService persistenceService) {
31         this(delegate, identityProvider, new ArchiveVisitor(persistenceService), operationOverride: null);
32     }
33
34     public ArchiverLogger(Archiver delegate,
35                          AppUserService identityProvider,
36                          ArchiveVisitor visitor,
37                          String operationOverride) {
38         this.delegate = delegate;
39         this.identityProvider = identityProvider;
40         this.visitor = visitor;
41         this.operationOverride = operationOverride;
42     }
43
44     @Override
45     public void compress(Path sourceDir, Path targetArchive) throws IOException {
46         AppUser user = identityProvider.resolveCurrentUser();
47         long started = System.nanoTime();
48
49         ArchiveMetadataUtil ArchiveMeta meta = ArchiveMetadataUtil.detect(targetArchive);
50
51         ArchiveOperation ctx = new ArchiveOperation();
52         ctx.setUser(user);
53         ctx.setOperation(operationOverride != null ? operationOverride : "COMPRESS");
54         ctx.setArchivePath(targetArchive.toAbsolutePath().toString());
55         ctx.setTargetPath(targetArchive.toAbsolutePath().toString());
56         ctx.setFormat(meta.format());
57         ctx.setCompression(meta.compression());
58         ctx.setChecksumType(meta.checksumType());
59
60         try {
61             delegate.compress(sourceDir, targetArchive);
62
63             ctx.setStatus(status: "SUCCESS");
64             ctx.setSizeBytes(Files.size(targetArchive));
65             ctx.setChecksumValue(ArchiveMetadataUtil.sha256(targetArchive));
66         }
```

Рис. 2 ArchiverLogger частина 1

Висновок

У ході цієї лабораторної роботи я закріпив теоретичні знання про шаблони «Abstract Factory», «Factory Method», «Memento», «Observer» та «Decorator» і на практиці застосовував їх у контексті проектування програмної системи. Я створив власну ArchiverFactory з використанням фабричних підходів для уніфікованого створення архіваторів, а також розробив декоратор ArchiverLogger, який додає до архіваторів функціональність логування без зміни їхньої основної логіки. Крім цього, була побудована узагальнена діаграма класів, яка демонструє взаємодію ключових компонентів системи. Загалом робота показала, що застосування шаблонів проектування дозволяє робити систему більш структурованою, гнучкою та зручною для подальшого розширення.

Контрольні запитання

1. Яке призначення шаблону «Абстрактна фабрика»?

(Створення сімейств взаємопов'язаних об'єктів без прив'язки до їх конкретних класів)

2. Нарисуйте структуру шаблону «Абстрактна фабрика».

(Складається з інтерфейсу фабрики, конкретних фабрик, інтерфейсів продуктів та конкретних продуктів. Клієнт взаємодіє тільки з абстракціями)

3. Які класи входять в шаблон «Абстрактна фабрика», та яка між ними взаємодія?

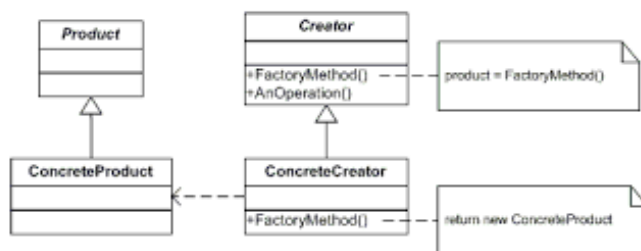
- ✧ AbstractFactory — оголошує методи створення продуктів.
- ✧ ConcreteFactory — реалізує створення конкретних продуктів.
- ✧ AbstractProduct — інтерфейс продукту.
- ✧ ConcreteProduct — конкретні реалізації продуктів.

(Взаємодія: клієнт використовує фабрику для отримання необхідних об'єктів, не знаючи їх конкретних типів)

4. Яке призначення шаблону «Фабричний метод»?

(Дозволяє підкласам визначати, який об'єкт створювати, через перевизначення фабричного методу)

5. Нарисуйте структуру шаблону «Фабричний метод».



6. Які класи входять в шаблон «Фабричний метод», та яка між ними взаємодія?

- ✧ Creator — містить `factoryMethod()`.
- ✧ ConcreteCreator — перевизначає метод і створює конкретний продукт.
- ✧ Product — інтерфейс продукту.
- ✧ ConcreteProduct — реалізація продукту.

(Взаємодія: клієнт викликає метод створення у Creator, а конкретний продукт визначається підкласом)

7. Чим відрізняється шаблон «Абстрактна фабрика» від «Фабричний метод»?

Абстрактна фабрика створює сімейства об'єктів, фабричний метод — один об'єкт.

Абстрактна фабрика — складніший шаблон.

Фабричний метод розширюється через нові підкласи Creator, абстрактна фабрика — через нові фабрики.

8. Яке призначення шаблону «Знімок»?

(Зберігати стан об'єкта і дозволяти відновлювати його, не порушуючи інкапсуляції)

9. Нарисуйте структуру шаблону «Знімок».

(Відповідь) Є три учасники: Originator (створює знімок), Memento (зберігає стан), Caretaker (зберігає знімки).

10. Які класи входять в шаблон «Знімок», та яка між ними взаємодія?

Originator — створює і відновлює Memento.

Memento — зберігає внутрішній стан.

Caretaker — управляє колекцією знімків, але не знає їх змісту.

(Взаємодія: Caretaker зберігає знімки, Originator читає/пише стан)

11. Яке призначення шаблону «Декоратор»?

(Додавати нову поведінку об'єктам динамічно, не змінюючи їхніх класів)

12. Нарисуйте структуру шаблону «Декоратор».

(Є базовий компонент, конкретний компонент, декоратор (який містить компонент) та конкретні декоратори, що додають функціонал)

13. Які класи входять в шаблон «Декоратор», та яка між ними взаємодія?

- ✧ Component — базовий інтерфейс.
- ✧ ConcreteComponent — основний об'єкт.
- ✧ Decorator — містить посилання на компонент.
- ✧ ConcreteDecorator — додає поведінку.

(Взаємодія: об'єкт обгортається в декоратор(и), які розширюють його можливості)

14. Які є обмеження використання шаблону «Декоратор»?

- ✧ Велика кількість дрібних класів.
- ✧ Складність дебагу при багатьох рівнях обгортання.
- ✧ Неможливість зміни інтерфейсу компонента.
- ✧ Порядок декорування впливає на поведінку.